Documentación Complementaria - HumedApp

Este documento contiene los puntos faltantes a partir del 3.6, redactados según el formato de la universidad y basados en la arquitectura real del proyecto en React.

3.6 Restricciones y Atributos de Calidad

Atributos de Calidad

1. **Usabilidad:** La aplicación cuenta con una interfaz intuitiva y responsiva (desarrollada con Tailwind CSS), orientada al uso en dispositivos móviles mediante una barra de navegación inferior (*Bottom Navigation*). Los colores y la tipografía respetan un diseño visual ecológico (tonos esmeralda y tierra) que facilita la lectura y accesibilidad en entornos exteriores o con alta luz solar.
2. **Rendimiento:** Está construida como una Single Page Application (SPA) usando React 19 y Vite, lo que asegura tiempos de carga mínimos y transiciones fluidas entre los componentes de visualización y monitoreo.
3. **Disponibilidad y Concurrencia:** La persistencia de datos (como reportes ambientales) y la gestión de usuarios se maneja a través de Supabase (Backend-as-a-Service), asegurando alta disponibilidad a través de la infraestructura cloud global, y permitiendo la creación de cuentas concurrentes sin degradación del servicio.
4. **Seguridad:** Las políticas de Row Level Security (RLS) en Supabase restringen el acceso y la mutación de los datos. Solo usuarios autenticados pueden generar o eliminar sus propios reportes (Incidentes Ambientales). Adicionalmente, las credenciales del API de Google Gemini y Supabase se administran mediante variables de entorno en tiempo de compilación.

Restricciones

1. **Hardware / Plataforma:** La aplicación está diseñada bajo el ecosistema web móvil, soportado nativamente en navegadores modernos e instanciable como una Progressive Web App (PWA) o contenedor WebView en Android (requerimiento prioritario del proyecto).
2. **Dependencias Externas Restrictivas:**
   - **Google Gemini API:** El módulo de educación interactiva (*Chatbot HumedBot*) requiere conexión estable a internet y depende de las cuotas de uso y latencia provistas por la infraestructura de Google Cloud.
   - **Sensores de Hardware (GPS y Cámara):** El módulo de Participación / Monitoreo está estrictamente limitado a que el usuario otorgue explícitamente permisos a las APIs del navegador (`navigator.geolocation` y captura de medios) para la correcta georreferenciación y recolección de evidencia multimedia.

---

4. Diseño del software (ISO -12207-1)

4.1 Diseño detallado del software

La aplicación sigue una arquitectura basada en Componentes de React, gestionando el estado global de autenticación (`AuthContext`) y enrutando vistas lógicas a través del componente principal (`App.tsx`).

4.1.1 Diagrama de clases (Lógico / Interfaces)

Basado en el sistema de tipos de TypeScript (`types.ts`), las entidades principales del sistema se modelan de la siguiente forma (descripción para modelado UML):

- **Enumeración `AppTab`:** Controla la navegación principal (`HOME` , `MONITORING` , `EDUCATION` , `PARTICIPATION` , `CULTURE` , `CHAT`).
- **Clase/Interfaz `EnvironmentalReport`:** Representa una denuncia o reporte ciudadano. Atributos: `id` (numérico), `type` (fauna, flora, emergencia), `title` (texto), `description` (texto), `coords` (tupla numérica lat/lng), `imageUrl` (cadena), `timestamp` (fecha), `ownerId` (UUID Supabase).
- **Clase/Interfaz `UserProfile`:** Maneja la gamificación y el estatus. Atributos: `name` , `bio` , `level` (Guardián, etc.), `points` , `reportsCount` .
- **Clase/Interfaz `Species`:** Entidad del catálogo biológico. Atributos: `id` , `name` , `scientificName` , `category` (ave, planta, mamífero, anfibio), `description` , `imageUrl` .

4.1.2 Diagrama de componentes

La vista estructural agrupa el código fuente en los siguientes módulos fundamentales (representables en un diagrama de componentes):

- **Componente de Enrutamiento Base (`App.tsx`):** Orquesta los `AuthScreen` , `Dashboard` , `Monitoring` , etc.
- **Módulo de Interfaz Lógica (`/components`):** Contiene la lógica visual de las vistas (e.g. `Dashboard.tsx` , `Chatbot.tsx` , `Navigation.tsx`).
- **Módulos de Gestión de Estado (`/components/AuthContext.tsx`):** Proveedor global del estado de sesión y wrapper de Supabase Auth.
- **Módulos de Integración Externa (`/services`):**
  - `supabaseClient.ts` : Capa de transporte y conectividad con la base de datos relacional (PostgreSQL).
  - `geminiService.ts` : Controlador de las peticiones HTTP/SDK hacia el modelo LLM de Google.

4.1.3 Diagrama de actividades

Ejemplo de Flujo Principal (Reportar Riesgo Ambiental):

1. **Inicio:** El usuario (estando autenticado) ingresa al módulo de Monitoreo (`AppTab.MONITORING`).
2. **Interacción:** El sistema solicita acceso a la ubicación GPS.
3. **Captura de Evidencia:** El usuario activa la cámara nativa en la UI, captura una fotografía de un vertimiento o quema, e ingresa Título y Descripción.
4. **Procesamiento:** El sistema sube la imagen (ya sea usando un storage provider o base64) y empaqueta el JSON del reporte (`EnvironmentalReport`).
5. **Persistencia:** Se envía la solicitud a Supabase mediante una mutación.
6. **Fin:** El sistema renderiza un nuevo marcador (pin rojo/naranja) en el mapa interactivo (Leaflet) para que los demás ciudadanos puedan visualizarlo en tiempo real.

4.1.4 Diagramas de despliegue

1. **Capa Cliente (Dispositivo Android):** Navegador Web Móvil / WebView que ejecuta el bundle de JavaScript (generado por Vite).
2. **Capa del Servidor Web (Hosting Estático):** Servidor CDN (como Vercel, Netlify o GitHub Pages) responsable de entregar los archivos `.html` , `.js` y `.css` .
3. **Capa Backend BaaS (Supabase Hub):** Instancia concurrente auto-gestionada que expone una Data API (REST/GraphQL) y el flujo OAuth para usuarios (PostgreSQL Database).
4. **Capa Externa API (Google Cloud):** Nodo de Google AI Studio para procesamiento de peticiones PNL (Gemini).

4.2 Diseño de la Interfaz

4.2.1 Interfaz de Usuario (Hardware, Software y Comunicaciones)

La interfaz se ha implementado mediante componentes declarativos en React. Se asume un *viewport* de celular (`max-w-md mx-auto` en Tailwind). Usa iconos vectoriales (librería `lucide-react`) para garantizar nitidez visual en pantallas de alta densidad de pixeles (Retina/OLED).

4.2.2 Interfaz de Entrada (Formularios, Inicio de Sesión)

- **Pantalla de Autenticación (`AuthScreen.tsx`):** Formulario unificado para `Login` y `Registro` . Requiere validación asíncrona de sintaxis de correo y contraseñas seguras.
- **Formularios de Reportes:** Entradas de texto controlado (React states), botones para `startCamera()` , y dropdowns o selectores tipo pastilla (*pills*) para la categorización del reporte (`flora` , `fauna` , `riesgo`).

4.2.3 Interfaz de Salida (Reportes, Consultas, Impresiones)

- **Mapa Interactivo (`Monitoring.tsx`):** Lienzo del mapa usando la librería Leaflet renderizando *TileLayers* de OpenStreetMap y renderizando un array de coordenadas a través de *Markers* dinámicos con popups de visualización.
- **Chat Conversacional (`Chatbot.tsx`):** Interfaz asíncrona tipo mensajería, con burbujas de texto diferenciadas visualmente entre el usuario y el bot.

4.3 Diseño del Modelo de Datos / Persistencia

La persistencia se asienta sobre Supabase, que envuelve una base de datos PostgreSQL.

- **Esquema `auth.users` :** (Gestionado por Supabase) Guarda las identidades de acceso.
- **Tabla `reports` :** Almacena la telemetría ciudadana. Columnas proyectadas: `id` (uuid, PK), `created_at` (timestamp), `user_id` (FK a auth.users), `coords_lat` (float), `coords_lng` (float), `description` (text), `category` (varchar), `image_url` (varchar).
- **Tabla `profiles` :** Integrado con triggers, almacena la información social (`bio` , `level` , `points`).

4.4 Diseño de la Arquitectura de Software (Modelo C4)

- **Nivel Contexto:** El sistema HumedApp interactúa con el Ciudadano, el Administrador (roles de Supabase) y con el agente de Inteligencia Artificial (Google).
- **Nivel Contenedores:** Separación principal entre: App Móvil (React/Vite), Backend y Base de Datos Integrada (Supabase), Autenticación (Supabase Auth).
- **Nivel Componentes:** Dentro de la App Móvil conviven los controladores de estado visual (Vistas como Education, Culture, Dashboard) y los Servicios Integradores (GeminiService.ts, supabaseClient.ts).

---

5. Implementación

5.1 Herramientas Utilizadas en el Desarrollo del Proyecto

- **Entorno de Ejecución:** `Node.js` v22 (compilación).
- **Framework Core:** `React` v19 (Interfaz) con `TypeScript` v5.8 (Tipado estricto).
- **Construcción y Empaquetado:** `Vite` v6 (Servidor local ultrarrápido y Hot Module Replacement).
- **Estilos y CSS:** `Tailwind CSS` (Framework de utilidad primaria) garantizando animaciones (*@keyframes fadeIn*) y control responsivo.
- **Mapas y Geolocalización:** `Leaflet` (1.9.4) integrado nativamente.
- **Servicios Cloud:** `@supabase/supabase-js` (Baas, API Key config) y `@google/genai` (Inteligencia Artificial LLM).

- **Iconografía:** `lucide-react`.

5.2 Requisitos del Hardware

Dado el enfoque *mobile-first / web*, el sistema exige las siguientes especificaciones recomendadas del lado del cliente:

- Smartphone (Android 8.0 o superior, iOS 14+) con conectividad 3G/4G o WiFi.
- Módulo GPS asistido o acelerómetro (Permiso de geolocalización esencial).
- Cámara nativa integrada (Permiso de acceso a medios para *capturas ambientales*).
- Al menos 2GB de RAM disponible para renderizar sin interrupciones los tiles topográficos del mapa Leaflet.

---

6. Pruebas del Software

6.1 Inspección de Software (Validación y Verificación)

1. **Pruebas de Componente (React JSX):** Verificación técnica de los estados nulos en carga, por ejemplo, asegurar que `AuthContext` retorna los *spinners* o mensajes de `Cargando sesión...` adecuadamente, evitando desbordamientos o renderizados vacíos.
2. **Prueba de Flujo de Datos (Endpoints):** Ejecución de mutaciones simuladas y *mocking* hacia Supabase para confirmar que la fila de "Reporte Ambiental" se inserta y se lee dentro del radio de geovalla (Localidad de Kennedy).
3. **Verificación de Modelo de Lenguaje:** Control del Sandbox de *Gemini*; someter prompts negativos o consultas fuera de la biología (ej. política, economía) para asegurar que el modelo responda únicamente bajo el dominio ecológico de los humedales.

6.2 Pruebas de Usabilidad – Resultados

Se aplicarán heurísticas de Nielsen sobre el dispositivo.

- **Heurística de Visibilidad y Estado:** El uso del componente `<UserProfilePanel>` y un contador de notificaciones asegura que el usuario comprende si su queja ha sido enviada exitosamente o si se requiere iniciar sesión (pantalla `AuthScreen`).
- **Prevención de Errores:** Botones de acción principales (como capturar foto) cuentan con *disabled states* si el dispositivo carece de cámara o el GPS no logra fijar coordenadas a tiempo, mostrando diálogos descriptivos en lugar de errores crípticos del navegador.

6.3 Modificaciones Realizadas

El proyecto fue migrado de un concepto estático a un concepto React/Vite reactivo.

- Se refactorizó el modelo de un único HTML gigante hacia múltiples componentes inyectados dinámicamente según la variable `activeTab` del estado principal (`useState<AppTab>`).
- La lógica de validación se extrajo hacia sus propios contextos (`AuthContext.tsx`), permitiendo escalar de forma limpia para futuras versiones que agreguen roles administrativos.

Conclusiones y Recomendaciones

Conclusiones

- La integración armónica de tecnologías ágiles y modernas (React, TypeScript, Vite) demostró ser un pilar crítico para construir una experiencia de alta fidelidad, compitiendo técnicamente con una app móvil nativa, pero conservando los beneficios de una actualización web unificada.

- El uso del ecosistema Supabase abstrae la complejidad del mantenimiento tradicional de un backend, acelerando la capacidad del proyecto de concentrarse en ofrecer valor específico: la red de monitoreo georreferenciada.
- La inserción de agentes impulsados por IA (*Gemini*) en contextos de educación ambiental potencia la divulgación interactiva rompiendo con la enciclopedia estática; resultando en una apropiación social del territorio más robusta e inmediata.

Recomendaciones

- **Ampliación del modo Offline:** A pesar del empaquetado inicial, se sugiere incorporar Service Workers integrales (`workbox`) para lograr un caché persistente de la red vial cartográfica del humedal, permitiendo que la navegación e interacción se mantenga en zonas profundas sin cobertura 4G.
- **Hardware Complementario (IoT):** A nivel físico y en asociación con la universidad, el despliegue de sondas LoRaWAN o sensores de bajo consumo de red ayudaría a alimentar de forma automática el módulo `Monitoring.tsx` (que en su primera fase requiere alimentarse de la percepción del ciudadano).
- **Soporte Multi-Rol:** Desarrollar en un segundo Sprint el panel de un "Administrador Municipal / Ambiental" (Validado desde los RLS de Supabase) para filtrar casos o emitir las Alertas (Políticas Públicas).

Bibliografía

1. Facebook Open Source. (2025). *React Documentation: A JavaScript library for building user interfaces*.
2. Typescript Lang. (2025). *TypeScript Docs: JavaScript with syntax for types*.
3. Leaflet. (2024). *Leaflet: an open-source JavaScript library for mobile-friendly interactive maps*.
4. Supabase. (2025). *The open source Firebase alternative: Architecture and Edge Functions*.
5. Google Cloud / Gemini. (2025). *Google GenAI SDK Documentation / Prompt Engineering Guidelines*.
6. Alcaldía Mayor de Bogotá – Secretaría de Ambiente. (2020). *Informe técnico de Humedales Urbanos: Techo y El Burro*.

Anexos

- [Anexo A] Repositorio de Código Fuente de HumedApp (React/TypeScript).
- [Anexo B] Diagramas UML en sintaxis PlantUML (`plantuml/*.uml`).
- [Anexo C] Archivo `.env.local` (*Plantilla* con los accesos al proveedor).

Manual de Usuario (Aplica especialmente para Administradores de Pruebas y Revisores)

1. Instalación y Configuración del Entorno Local

1. **Prerrequisitos:** Asegúrese de tener instalado Node.js (v20 o superior).
2. **Clonado del Repositorio:**

```
git clone https://.../humedapp.git
cd humedapp---humedal-de-techov3
```

3. **Instalación de Dependencias:** Ejecute `npm install` en la ruta raíz para descargar bibliotecas base como `lucide-react`, `@google/genai`, y `leaflet`.
4. **Configuración de Variables de Entorno (`.env.local`):** En la raíz del proyecto, cree o edite el archivo `.env.local` agregando sus credenciales provistas por la universidad:

```
VITE_SUPABASE_URL=aqui_va_su_url_de_supabase
VITE_SUPABASE_ANON_KEY=aqui_va_su_token_anon
VITE_GEMINI_API_KEY=aqui_va_su_apikey_genai_google
```

5. **Ejecución:** Levante el servidor de desarrollo Vite ejecutando `npm run dev`. El sistema mostrará un puerto local (usualmente `http://localhost:5173/`).

2. Uso Funcional de la Aplicación

1. **Ingreso a la Plataforma:** Al acceder por el navegador en dispositivo móvil u ordenador, se visualizará inmediatamente el *Dashboard Principal* (resumen ambiental).
2. **Autenticación Requerida:** Para acceder al menú secundario (Monitoreo mediante Mapa Interactivo) y la red de Participación Comunitaria, el sistema exigirá iniciar sesión.
   - *Nota:* Utilice una cuenta de correo de prueba en el componente "Crear cuenta".
3. **Módulo de Monitoreo (El Mapa):** Navegue al icono del *Centro* (Mapa/Radar). Un aviso le pedirá que otorgue el permiso a su **Ubicación GPS**. Si no lo entrega, no podrá visualizarse a usted mismo en el campo. Seguidamente, dé clic en el botón flotante (+) para activar su cámara y remitir la denuncia respectiva ante un vertimiento ilegal o el avistamiento de un ave.
4. **Educación Interactiva (HumedBot):** En la cinta inferior, ingrese al módulo conversacional (burbuja de chat). Establézcale preguntas directamente al agente como: *¿Por qué son importantes los humedales del suroccidente de Bogotá?*. Él contestará pedagógicamente basándose en las APIs integradas.
5. **Perfil y Puntuación:** Accediendo en el ícono superior derecho ("Menú Hamburguesa") desplegará la interfaz de Nivel. Las interacciones positivas, lecturas de las enciclopedias botánicas en el módulo *Educación*, y subida de incidentes sumará a su escalafón ciudadano.